

# La revolución de Othello, Bothello

Francisco Gago Vázquez

Dpto. Ciencias de la Computación e Inteligencia Artificial  
Universidad de Sevilla  
Sevilla, España  
fragagvaz@alum.us.es

Marco Antonio Herrera Luján

Dpto. Ciencias de la Computación e Inteligencia Artificial  
Universidad de Sevilla  
Sevilla, España  
marherluj@alum.us.es

**Resumen**—Este trabajo tiene como objetivo construir un agente dotado de Inteligencia artificial para el juego de mesa de “Othello”, el cual sea capaz de tomar decisiones estratégicas y estudiar su efectividad en el juego. Para ello, se han empleado herramientas modernas de inteligencia artificial, en concreto una combinación entre el algoritmo de búsqueda en árboles de Monte Carlo (MCTS) utilizando un enfoque de Upper Confidence Bound Tree (UCT) combinado con la adición de aprendizaje supervisado por el uso de una Red Neuronal que es capaz de inferir la probabilidad de victoria dada una posición en el juego.

Tras haber desarrollado agentes de distintos tipos, y haber estudiado su porcentaje de victorias respecto al de los otros modelos, hemos llegado a la conclusión de que el agente MCTS es mejor que un agente que implemente una red neuronal que no esté bien entrenada.

**Palabras clave**—Inteligencia Artificial, Aprendizaje por Refuerzo, Aprendizaje Supervisado, Red Neuronal, Monte Carlo Tree Search, Othello, Reversi, Python, Pygame, Upper Confidence Bound Tree.

## I. INTRODUCCIÓN

El siguiente trabajo se ha desarrollado en un entorno académico, con el objetivo de mostrar el conocimiento aprendido por los miembros del grupo durante la asignatura “Inteligencia Artificial” para la evaluación del mismo, por lo que se han empleado algunas de las técnicas y algoritmos vistos en clase para resolver el problema planteado por el profesor. El trabajo está centrado en el universo del juego de mesa “Othello”, el cual hemos modelado en el lenguaje de programación Python con ayuda de la librería “Pygame” [1]-[2] apoyándonos en tutoriales de YouTube para realizar el código tanto del juego [3]- [5] como del menú [6]- [7] mostrado en (Fig. 1.). Una vez realizado el código tanto del juego como del menú, ya podíamos jugar una persona contra otra persona, pero, para acercarnos a nuestro objetivo de incluir agentes inteligentes, desarrollamos primero un bot que realizara jugadas aleatorias y a su vez desarrollamos la lógica necesaria para que al jugar contra el bot, la partida quedara registrada en un fichero csv con el texto completamente preparado con vista a entrenar una red neuronal en el futuro.

Una vez teníamos el bot y el sistema de generación de datos, nos informamos sobre cómo deberíamos implementar un agente [8] para cumplir con el objetivo de atacar un problema de búsqueda adversaria en un entorno competitivo, en concreto, en un juego determinista, de dos jugadores, por turnos, con información perfecta y de suma cero (es decir,

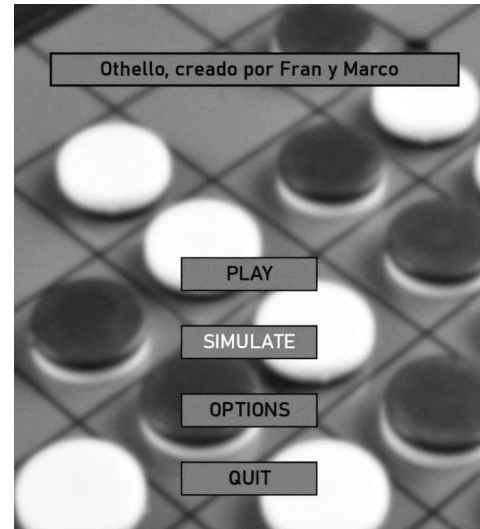


Fig. 1. Imagen del menú en Pygame.

lo que es bueno para un jugador es un inconveniente para su oponente).

Para crear el agente final, primero creamos un agente sin aprendizaje supervisado basándonos en el algoritmo de búsqueda en árboles de Monte Carlo (MCTS). Para construirlo, nos basamos en el pseudocódigo descrito en la investigación de Cameron B. Browne et al. sobre los métodos de búsqueda en árbol de Monte Carlo [9] en el que se equilibran las fases de exploración de nodos que no tienen una fiabilidad alta sobre su resultado con la de explotación de nodos que sí han dado buenos resultados anteriormente.

Por último, una vez terminamos el agente que implementa el algoritmo de Monte Carlo, lo pusimos a jugar contra sí mismo para generar partidas que alimentaran a la futura red neuronal y así poder usarla para sustituir el resultado que nos da la red dado un estado del tablero por la búsqueda por defecto que realiza el algoritmo de Monte Carlo.

A lo largo del resto del documento, se describe el transcurso del proyecto, definiendo las técnicas empleadas y los trabajos relacionados en los preliminares, los métodos y algoritmos utilizados en la sección de metodología, los resultados obtenidos de la aplicación de algoritmos y métodos en la sección de resultados, el aprendizaje obtenido de los resultados y su estudio en la sección de conclusiones y por último las

secciones de bibliografía y consultas a IA generativa.

A continuación presentamos una breve tabla de contenido sobre el presente documento:

- 1) Introducción
- 2) Preliminares
  - a) Métodos empleados
  - b) Trabajo Relacionado
- 3) Metodología
  - a) Preparación del proyecto
  - b) Implementación de Otelo
  - c) Habilitación de bot
  - d) Creación del agente
  - e) Grabación de partidas
  - f) Simulación de partidas
  - g) Creación de la red neuronal
- 4) Resultados
  - a) Bot vs Bot
  - b) Agente MCTS vs Bot
  - c) Agente MCTS vs Agente MCTS
  - d) Red Neuronal vs Bot
  - e) Red Neuronal vs Agente MCTS
  - f) Red Neuronal 1 vs Red Neuronal 2
- 5) Conclusiones
- 6) Uso de la IA generativa
- 7) Bibliografía

## II. PRELIMINARES

Durante la realización del proyecto hemos utilizado dos técnicas principales: el algoritmo de búsqueda en árboles de Monte Carlo (MCTS) con enfoque Upper Confidence Bound Tree (UCT) y el modelo de Inteligencia Artificial basado en aprendizaje supervisado de las redes neuronales, tal y como se ha utilizado en proyectos similares con anterioridad, por ejemplo en juegos de mesa como el “Go” [10] parece que esta aproximación es la que mejor funciona debido a la gran complejidad que posee el juego y que por tanto hace imposible la búsqueda de un camino óptimo.

### A. Métodos empleados

#### 1) Upper Confidence Bound Tree

Nuestra aproximación para crear un agente capaz de hacer movimientos de forma estratégica a lo largo de la partida de Otelo, se basa en el uso del algoritmo de búsqueda en árboles de Monte Carlo (MCTS) con un enfoque Upper Confidence Bound Tree (UCT) como método para equilibrar la exploración y la explotación. La fórmula de UCT es la siguiente:

$$UCT = \bar{X}_j + 2C\sqrt{\frac{2\ln n}{n_j}}$$

donde:

- $\bar{X}_j$ : es la recompensa del nodo  $j$  dividida entre la cantidad de veces que ha sido visitado.

- $C$ : es una constante que permite regular entre el término exploratorio y el término explotador de la función UCT.
- $n$ : es el número de veces que el nodo actual (nodo padre) ha sido visitado.
- $n_j$ : es el número de veces que el hijo  $j$  ha sido visitado.

La forma en la que  $C$  sirve para controlar el equilibrio entre exploración y explotación es la siguiente:

- $\bar{X}_j$ : se corresponde con el término explotador, es decir, cuanto mayor sea la recompensa acumulada por entrar al nodo, más probable será que se vuelva a visitar y cuantas más veces se haya visitado el nodo, más se favorecerá ir a otros nodos que no se hayan visitado tantas veces.
- $C\sqrt{\frac{2\ln n}{n_j}}$ : es el término exploratorio, el cual favorece que se visiten nodos que no han sido visitados tantas veces (por lo que no se sabe aún si darán buenas recompensas o no). El término disminuye cuanto mayor sea  $n_j$ , es decir, cuanto más se haya explorado ese nodo, mientras que el logaritmo del numerador asegura que la exploración decrezca lentamente a medida que se visita al padre. La constante, por lo tanto, sirve para controlar la importancia que se le da a la exploración frente a la explotación.

#### 2) Monte Carlo Tree Search

El algoritmo de búsqueda en árboles de Monte Carlo ha probado ser una gran aproximación para encontrar la jugada óptima en juegos con información perfecta y de suma cero como el ajedrez o el Go.

Este algoritmo se basa en la realización de 4 pasos diferenciados como se muestra en la Fig. 1.:

- a) *Selección*: Se comienza en la raíz del árbol de búsqueda y si el nodo no está expandido se expande. Si está expandido, devolvemos el mejor hijo según el cálculo del UCT sobre todos sus hijos.
- b) *Expansión*: Teniendo un nodo no expandido, se elige una acción de las posibles acciones (es decir, jugadas) que se pueden realizar en ese estado del juego y se retorna el nuevo nodo. Si el nodo estaba expandido previamente, se llama a la función **best\_child**.
- c) *Simulación*: Ahora se realizan movimientos en un tablero simulado siguiendo la política pertinente. En un agente que implementara el MCTS sin modificaciones, se utilizaría la política por defecto o **default\_policy**, que en nuestro caso genera acciones de forma aleatoria hasta llegar a un nodo terminal. El agente que hemos implementado finalmente, sin embargo, utiliza una red neuronal para inferir la probabilidad de victoria o derrota dado ese nodo.
- d) *Retropropagación*: Una vez obtenido el resultado de la simulación, se actualizan los valores tanto de recompensa como de número de visitas por nodo.

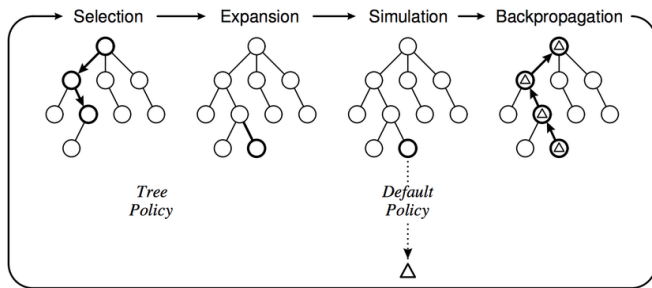


Fig. 2. Esquema de funcionamiento del algoritmo MCTS [11].

Todos estos pasos se realizan dado un “presupuesto computacional”, que normalmente es medido en tiempo (nuestro caso) o en número de iteraciones máximo. Nosotros hemos elegido otorgar 1 segundo para realizar los pasos anteriormente descritos.

### 3) Neural Network

La red neuronal es un modelo de Inteligencia Artificial basada en aprendizaje supervisado. Para utilizarla, creamos un módulo encargado de generar “datos de entrenamiento” dentro de nuestro proyecto. Cada vez que se ejecuta una partida, se puede activar una opción que guarda cada estado  $S$  de la partida con los siguientes datos:

- *casillas*: se corresponden con cada una de las casillas del tablero. Como es de 8 filas y 8 columnas, van desde  $a1, a2, \dots, h8$ . En cada casilla se encuentran los siguientes valores:
  - 0: si en esa casilla no hay ninguna pieza.
  - 1: si en esa casilla hay una pieza blanca.
  - 2: si en esa casilla hay una pieza negra.
- *current\_player\_won*: se corresponde con los siguientes valores:
  - 1: si el jugador que estaba jugando en ese turno terminó ganando la partida.
  - 0: si la partida terminó en empate.
  - -1: si el jugador que estaba jugando en ese turno terminó perdiendo la partida.

Tras hacer que el agente de Monte Carlo jugara contra sí mismo para generar datos suficientes, creamos una red neuronal que, dado un nuevo estado de la partida (es decir, el estado del tablero), infiere la probabilidad de ganar o perder la partida en el rango  $[-1, 1]$ .

### B. Trabajo Relacionado

El enfoque de utilizar el algoritmo de MCTS junto con una red neuronal para inferir la política no es algo novedoso, se ha investigado en profundidad sobre el tema.

En el artículo “*Mastering the game of Go with deep neural networks and tree search*” [10] se explica cómo se entrenó al programa **AlphaGo** para llegar, por primera vez en el juego (el cual es de bastante mayor complejidad que el Othello e incluso que el ajedrez) a derrotar al campeón mundial de Go y a todos

los programas anteriormente presentados para competir en el juego de mesa.

## III. METODOLOGÍA

La metodología de trabajo que hemos seguido a lo largo del desarrollo del proyecto ha sido la siguiente:

- 1) *Preparación del proyecto*: creamos el repositorio en GitHub y construimos el entorno virtual de Python para el proyecto. Todas las dependencias necesarias se encuentran en el fichero `requirements.txt`.
- 2) *Implementación de Othello*: utilizando la librería Pygame, implementamos el juego de mesa Othello, consiguiendo que dos personas puedan jugar entre sí.
  - a) *Inicialización de juego*: creamos el módulo `main` permitiendo abrir el juego y mostrar el tablero, y los módulos `game_const`, que establece constantes básicas del juego, como las dimensiones de la pantalla o el tamaño del tablero y `game_var`, que establece parámetros variables dentro del juego.
  - b) *Colocación de piezas*: implementamos la clase `Board` y la clase `Piece`, con las funcionalidades necesarias para que los jugadores colocaran una ficha al clicar en cada casilla del tablero.
  - c) *Aplicación de reglas de Othello*: desarrollamos toda la lógica necesaria para ceñirnos a las reglas de Othello, como la captura de piezas del oponente, la validación de movimientos o el cambio de turno.
- 3) *Habilitación de bot*: programamos el juego para permitir partidas contra un bot controlado por el sistema, el cual escoge sus movimientos de forma completamente aleatoria.
  - a) *Incorporación de modos de juego*: como complemento a la creación del bot, añadimos la posibilidad de seleccionar entre distintos modos de juego. De este modo, el usuario puede optar por jugar contra otro jugador, enfrentarse al bot, o bien observar una partida controlada íntegramente por el sistema.
- 4) *Creación del agente*: incorporamos un agente capaz de tomar decisiones estratégicas utilizando el algoritmo Monte Carlo Tree Search (MCTS) siguiendo el pseudocódigo explicado en (Fig. 3.) basado en el explicado en [9].
  - a) *Enfoque UCT*: se utilizó la variante Upper Confidence Bound Tree (UCT) con el fin de equilibrar de manera efectiva la exploración y la explotación durante la búsqueda. Esto permite que el algoritmo seleccione tanto las acciones con mejores resultados históricos como aquellas opciones menos exploradas.
  - b) *Elección de rol de ambos jugadores*: mejoramos la selección de modos de juego añadiendo la selección independiente del controlador asignado a cada jugador (humano, bot o agentes con distintas

políticas). De este modo, el usuario puede decidir quién controla las fichas negras y quién las blancas, habilitando así todas las combinaciones posibles entre tipos de jugador.

- 5) *Grabación de partidas*: añadimos una funcionalidad para almacenar registros de las partidas jugadas en un fichero csv, de modo que posteriormente se pudiera utilizar para entrenar una red neuronal. Estos datos se pueden crear de manera opcional, en un archivo .csv que se genera en la carpeta `agent/training/games`, y toma el nombre según el tipo de partida que se grabe. Por tanto, si se juega una partida en la que un bot juega con las fichas negras y un agente en modo UCT (sin red neuronal) controla las blancas, se guardará en `agent/training/games/BOTvsAGENT_UCT.csv`. Posteriormente, todos estos ficheros se copiarán y se trasladarán a otro archivo que para entrenar la red.
- 6) *Simulación de partidas*: Para grabar datos de entrenamiento de manera automatizada, implementamos un sistema de generación de partidas automáticas. Con la generación de partidas, logramos dotar a la red neuronal de una base de datos sólida para su entrenamiento.
- 7) *Creación de la red neuronal*: Diseñamos una red neuronal capaz de predecir la probabilidad de victoria o derrota en función de la distribución del tablero. Esta red se entrena utilizando los datos generados por las partidas simuladas, lo que permite al agente aprender de las estrategias y patrones de juego. Para la creación de la red hemos utilizado un optimizador *SGD* y una estructura con:
  - Capa inicial: 64 neuronas (una por cada posición del tablero)
  - Capa intermedia: 32 neuronas y función de activación sigmoide
  - Capa final: 1 neurona y función de activación de tangente hiperbólica (para obtener resultado en el intervalo  $[-1, 1]$ )

Como hiperparámetros, hemos utilizado:

$$\eta = 0,01 \quad \text{MSE} = 1 \quad (1)$$

$$\text{epochs} = 800 \quad \text{batch} = 256 \quad (2)$$

- a) *Entrenamiento de la red*: El entrenamiento se realizó con un conjunto de datos de partidas simuladas, donde por cada turno se guardaba la disposición del tablero y el resultado en la partida del jugador protagonista en ese turno.
- b) *Separación de distintas redes*: Una vez entrenamos la red, almacenamos los resultados del entrenamiento en `bothello_first_training.h5`, con el fin de poder reutilizarla en el futuro. Esto nos permitió almacenar distintas versiones de la red y

poder enfrentarlas entre sí para ver cuál es la más efectiva.

- c) *Mejora continua de la red*: A partir de la primera red, generamos nuevas partidas y las utilizamos para entrenar una versión mejorada de la red. Este proceso es iterativo, y se podrá repetir tantas veces como se desee, con el fin de alcanzar un modelo mejorado que maximice la probabilidad de victoria.

#### IV. RESULTADOS

Tras implementar el Bot, realizamos algunos experimentos para valorar la calidad del agente una vez incorporada la red neuronal. Primero comenzamos por generar partidas entre los distintos "jugadores" que teníamos, en concreto obtuvimos los siguientes resultados:

##### 1) Bot vs Bot:

###### a) Configuración

Para este experimento contamos con la siguiente configuración:

- *Player 1*: Bot
- *Player 2*: Bot

###### b) Objetivo

En este experimento se quería determinar si dos jugadores que usaban siempre una acción aleatoria ganarían la mitad de las veces o no, por si en Ote lo existiera un factor que favoreciera la victoria como obtener el primer turno.

###### c) Resultados

Con una muestra de 200 partidas generadas, obtuvimos los siguientes resultados (Fig. 4.):

- *Victorias de Player 1*: 95
- *Victorias de Player 2*: 93
- *Empates*: 12

La conclusión es que simplemente colocando piezas al azar hay un 50% de probabilidad de ganar o perder.

##### 2) Agente MCTS vs Bot:

###### a) Configuración

Para este experimento contamos con la siguiente configuración:

- *Player 1*: Agente MCTS
- *Player 2*: Bot
- *Valor de constante C*:  $\frac{1}{\sqrt{2}}$

###### b) Objetivo

Tras haber implementado el algoritmo MCTS con UCT en el agente, queríamos probarlo con el que hemos tomado como el "jugador base", es decir, el bot que mueve aleatoriamente. A priori, pensamos que el Agente MCTS debería ganar un porcentaje muy alto de las partidas.

###### c) Resultados

Con una muestra de 200 partidas generadas, obtuvimos los siguientes resultados (Fig. 5.):

- *Victorias de Player 1*: 200

*búsqueda\_UCT()*

**Entrada:** un estado inicial y (opcionalmente) una red neuronal

**Salida:** una acción recomendada

- 1 crear nodo **raíz** con el estado inicial
- 2 establecer tiempo
- 3 mientras quede tiempo
- 4     seleccionar una hoja mediante la **política** del árbol
- 5     **si no hay red neuronal**
- 6         estimar recompensa simulando **aleatoriamente**
- 7     **si hay red neuronal**
- 8         estimar recompensa mediante la **política de la red**
- 9     **propagar** la recompensa hacia la raíz
- 10 devolver la **acción** asociada al mejor **hijo** de la raíz

*política\_del\_árbol()*

**Entrada:** un nodo del árbol

**Salida:** un nodo

- 1 mientras el **nodo no** sea **terminal**
- 2     si el nodo tiene acciones **sin explorar**
- 3         **expandir** el nodo y devolver el nuevo hijo
- 4     si todas las acciones ya han sido **exploradas**
- 5         avanzar al nodo hijo con mejor valor
- 6     devolver el nodo

*mejor\_hijo()*

**Entrada:** un nodo y la constante de exploración

**Salida:** el hijo con mayor valor de exploración

- 1 calcular el valor de cada hijo usando la fórmula **UCB**
- 2 devolver el hijo con mayor valor

*política\_aleatoria()*

**Entrada:** un estado de juego

**Salida:** una recompensa simulada

- 1 mientras el **estado no** sea **terminal**
- 2     elegir una acción **aleatoria**
- 3     **aplicar** la acción al estado
- 4 calcular y devolver la recompensa final

*política\_de\_red()*

**Entrada:** un estado y una red neuronal

**Salida:** una recompensa estimada

- 1 copiar y parsear el estado
- 2 estimar la **recompensa** de la red
- 3 devolver el valor de salida como recompensa estimada

*retropropagación()*

**Entrada:** un nodo hoja y una recompensa

- 1 mientras el **nodo no** sea **nulo**
- 2     **incrementar** el número de **visitas**
- 3     **acumular** la **recompensa**
- 4     **avanzar** al nodo padre

Fig. 3. Algoritmo de búsqueda UCT con red neuronal opcional

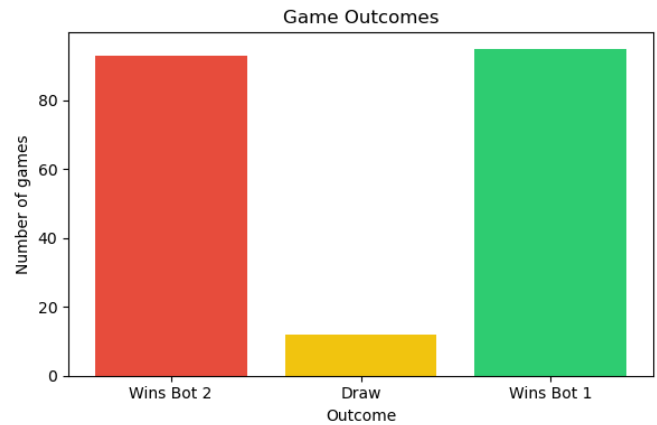


Fig. 4. Resultados de Bot contra Bot.

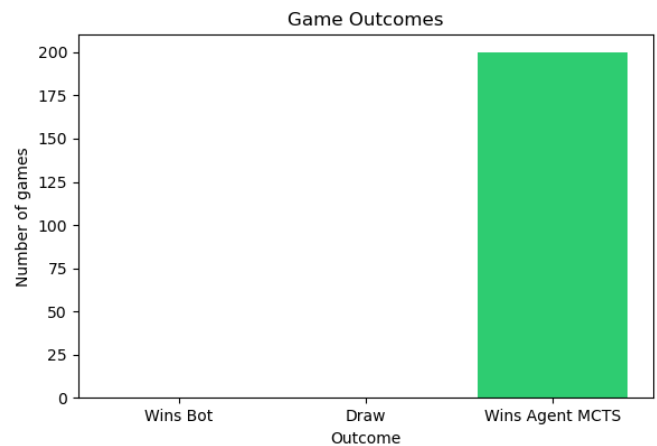


Fig. 5. Resultados de Agente MCTS contra Bot.

- Victorias de Player 2: 0
- Empates: 0

Como era de esperar, el agente gana todas las veces al bot, ya que sigue una estrategia mejor que jugar aleatoriamente para ganar.

### 3) Agente MCTS vs Agente MCTS:

#### a) Configuración

Para este experimento contamos con la siguiente configuración:

- Player 1: Agente MCTS
- Player 2: Agente MCTS
- Valor de constante  $C$  (para ambos):  $\frac{1}{\sqrt{2}}$

#### b) Objetivo

Este experimento pretendía comprobar qué ocurría al enfrentar al Agente MCTS contra sí mismo, a la vez que generar partidas para entrenar a la futura red neuronal. Esperábamos encontrar un porcentaje muy similar de victorias para cada jugador.

#### c) Resultados

Con una muestra de 200 partidas generadas, obtu-

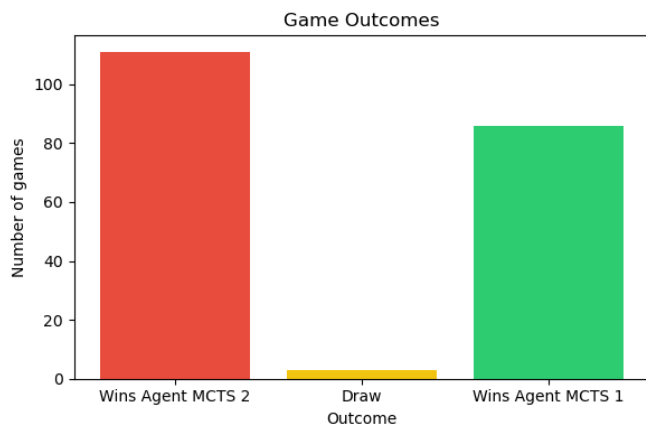


Fig. 6. Resultados de Agente MCTS contra Agente MCTS.

vimos los siguientes resultados (Fig. 6.):

- Victorias de Player 1: 86
- Victorias de Player 2: 111
- Empates: 3

Para sorpresa nuestra, encontramos que el Agente que jugaba en segundo turno tenía cierta ventaja, ya que cuantas más partidas generábamos más parecía acentuarse este comportamiento. Creemos que es debido a que un alto porcentaje de las partidas de Otelos son de un turno por cada jugador (teniendo en cuenta que si el otro no puede poner una ficha vuelve a ser tu turno), por lo que el jugador que termina la partida tendría a priori más fichas para volver de su color.

#### 4) Red Neuronal vs Bot:

##### a) Configuración

Para este experimento contamos con la siguiente configuración:

- Player 1: Red Neuronal
- Player 2: Bot
- Valor de constante  $C$ :  $\frac{1}{\sqrt{2}}$

##### b) Objetivo

Tras su primer entrenamiento, queríamos probar cómo se comportaba la red contra el Bot. Esperábamos que obtuviera un resultado mejor que el Bot, ya que el Bot ejecuta una acción completamente aleatoria.

##### c) Resultados

Con una muestra de 100 partidas generadas, obtuvimos los siguientes resultados (Fig. 7.):

- Victorias de Player 1: 47
- Victorias de Player 2: 50
- Empates: 3

Los resultados nos dieron a entender que la red neuronal era prácticamente igual al Bot en términos de porcentaje de victoria, lo que indica la necesidad de un entrenamiento mejorado.

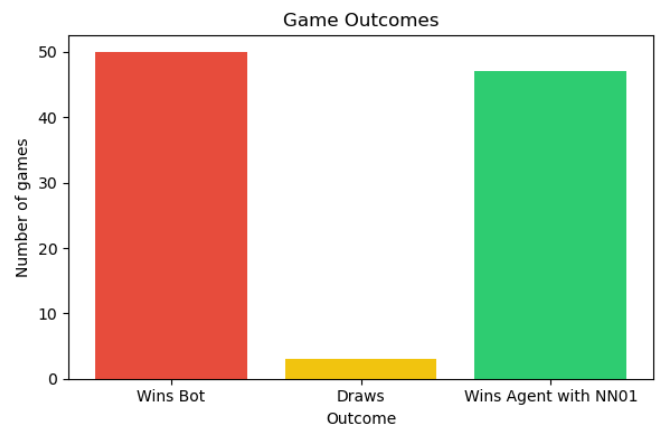


Fig. 7. Resultados de Agente con Red Neuronal 1 contra Bot.

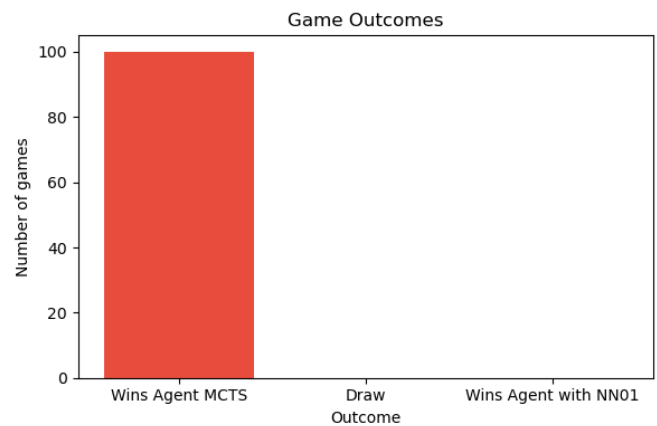


Fig. 8. Resultados de Agente con Red Neuronal 1 contra Agente MCTS.

#### 5) Red Neuronal vs Agente MCTS:

##### a) Configuración

Para este experimento contamos con la siguiente configuración:

- Player 1: Red Neuronal
- Player 2: Agente MCTS
- Valor de constante  $C$ :  $\frac{1}{\sqrt{2}}$

##### b) Objetivo

Tras haber probado la red neuronal contra el Bot, queríamos ver cómo se comportaba contra el agente, para ver si tenía más porcentaje de victoria que el Bot.

##### c) Resultados

Con una muestra de 100 partidas generadas, obtuvimos los siguientes resultados (Fig. 8.):

- Victorias de Player 1: 0
- Victorias de Player 2: 100
- Empates: 0

Los resultados no fueron nada positivos, lo que es una indicación clara de que debemos mejorar el en-

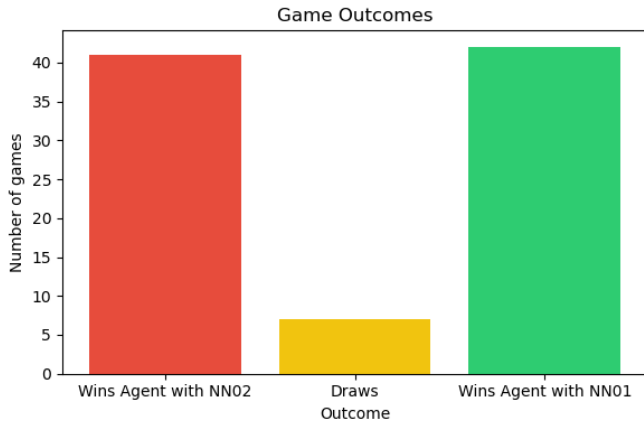


Fig. 9. Resultados de Agente con Red Neuronal 1 contra Agente con Red Neuronal 2.

trenamiento de la red para conseguir un porcentaje mayor.

#### 6) Red Neuronal 1 vs Red Neuronal 2:

##### a) Configuración

Para este experimento contamos con la siguiente configuración:

- Player 1: Red Neuronal 1
- Player 2: Red Neuronal 2
- Iteraciones para red 1: 800
- Iteraciones para red 2: 1500

##### b) Objetivo

Entrenamos a la red con más datos que la vez anterior (incluyendo las partidas que ella misma realizó contra el Bot y el Agente MCTS). Esperábamos que la red se comportara mejor que la vez anterior al haber recibido más datos y más iteraciones de entrenamiento.

##### c) Resultados

Con una muestra de 100 partidas generadas, obtuvimos los siguientes resultados (Fig. 9.):

- Victorias de Player 1: 42
- Victorias de Player 2: 41
- Empates: 7

Los resultados fueron algo decepcionantes, ya que tras entrenar de nuevo la red obtuvimos prácticamente el mismo resultado.

Los resultados anteriormente descritos se han resumido en la tabla I.

## V. CONCLUSIONES

Este trabajo ha sido nuestra primera aproximación a un trabajo real en el que se implementa la Inteligencia Artificial. Desde el principio estudiamos qué era lo que teníamos que realizar y dividimos el trabajo en varios hitos:

- Crear una versión funcional del juego
- Crear un Bot contra el que poder jugar

TABLA I  
RESUMEN DE RESULTADOS

| Jugador 1   | Jugador 2   | Victorias J1 | Empate | Victorias J2 |
|-------------|-------------|--------------|--------|--------------|
| Bot         | Bot         | 95           | 12     | 93           |
| Agente MCTS | Bot         | 200          | 0      | 0            |
| Agente MCTS | Agente MCTS | 86           | 3      | 111          |
| Agente NN01 | Bot         | 47           | 3      | 50           |
| Agente NN01 | Agente MCTS | 0            | 0      | 100          |
| Agente NN01 | Agente NN02 | 42           | 7      | 41           |

- Crear un Agente que implemente el algoritmo de MCTS con UCT
- Crear una Red Neuronal para sustituir **default\_policy** en el algoritmo MCTS

La versión del juego realizada en Pygame, a pesar del trabajo extra, resultó ser estética y funcionalmente mejor que la versión en consola. El primer bot no fue tan complicado de hacer como el hecho de poder tener dos jugadores independientes, aunque a la hora de implementar el algoritmo MCTS con enfoque UCT sí tuvimos más complicaciones (entender el algoritmo, implementar el código, entender cómo Python maneja self, etc.). Por último, con el entrenamiento de la red neuronal aprendimos de forma aproximada cómo se puede utilizar un modelo de aprendizaje supervisado en un proyecto real. También vimos que al enfrentarnos a un problema real era más complicado obtener buenas predicciones, ya que los conjuntos de datos vistos en clase estaban más preparados para obtener un buen resultado a diferencia de los nuestros al ser generados de manera automática durante el juego.

En cuanto a los resultados, algunos fueron predecibles, como los obtenidos de los Bots, aunque otros fueron más sorprendentes, en concreto aquellos realizados con los agentes y cómo algo que a priori puede estar oculto (ventaja para el jugador hace el segundo movimiento) se hace evidente realizando un análisis de este tipo.

Fue interesante leer estudios anteriores e informarse de los avances y la investigación realizada por otras personas. Estudiar y entender el algoritmo directamente de las fuentes [8]- [9] en lugar de otras fuentes a las que estamos más acostumbrados (YouTube, IA generativa) supuso un reto y tuvo como resultado un mayor aprendizaje del trabajo que estábamos realizando.

En cuanto al resultado general del proyecto, tenemos varias ideas de mejora futuras, como entrenar de mejor forma la red neuronal añadiendo valores al csv, como el jugador que está jugando en ese turno, o el número de piezas que tiene cada jugador, entre otras, y realizar modificaciones al módulo de generación de partidas y análisis de datos para que sea aún más robusto.

## VI. USO DE LA IA GENERATIVA

Durante el transcurso del proyecto, nos propusimos utilizar la Inteligencia Artificial Generativa para cuestiones realmente necesarias, enfocando nuestras fuentes a la hora de aprender a realizar cada una de las partes del trabajo en fuentes de

Internet (tutoriales, páginas web, documentación de librerías, etc.) y en las lecturas recomendadas por el profesor evaluador del trabajo. Todas estas fuentes están incluidas dentro de la bibliografía.

En cuanto a los prompts utilizados (todos a ChatGPT), fueron los siguientes:

- Quiero hacer un juego en pygame, y el tablero consiste de cuadrados separados por líneas negras. Me preguntaba cual podría ser la mejor forma de recrear el tablero, si usar un asset verde del color del tablero y pintar de alguna forma las líneas
- Crea una imagen de un asset para piezas de Reversi o Oteló, quiero una pieza blanca y una pieza negra en alta resolución sobre un fondo verde sólido, para posteriormente recortarlas
- Quiero modificar el valor de una constante que defino en otro archivo en mi juego de pygame en python. La importo a mi clase principal y cuando quiero cambiar su valor, obtengo lo siguiente:

UnboundLocalError: cannot access local variable 'MODE' where it is not associated with a value

- Estoy realizando un algoritmo basado en monte carlo tree search para jugar a un juego de mesa. Lo que no entiendo es cómo aprende montecarlo. Sé que va actualizando los nodos y guardando los valores, pero mi duda va hacia la implementación, ya que si lo guardo en memoria, cuando acabe la partida perdería toda esa información, por lo que no sé si esa información debería almacenarse de algún modo y, de ser así, cómo sería.
- Estoy teniendo problemas con esta clase en la que quiero implementar el método de búsqueda de montecarlo en árboles, creo que me he confundido con la diferencia entre nodo y estado, te adjunto la clase.
- Teniendo en cuenta lo que me dijiste, tengo algún problema con la clase Board, ya que a veces se ejecuta el tablero desde el punto de vista del agente y otras veces se ejecuta desde el punto de vista de la persona que hace el movimiento. He tenido varios problemas con el constructor, ya que a veces inicializa bien los valores, pero cuando quiero usar la clase con montecarlo tengo problemas. Te adjunto la clase.
- Te voy a enviar una red neuronal. La idea es que un agente la llame para que haga predicciones, pero no sé si esta es la forma correcta de hacerlo (organización de código, tener cuidado en que si hay dos agentes no se sobrescriban los datos de ambos objetos, etc)
- Puedes explicarme en mayor detalle qué es el upper bound confidence tree en el contexto de Monte Carlo Tree Search algorithm?
- Estoy haciendo un proyecto y quiero sacar gráficas de los distintos modelos que he puesto a jugar partidas. Te voy a adjuntar el ejemplo de una partida. Ayúdame a realizar en python con matplotlib funciones para visualizar el porcentaje de victoria de cada modelo.

## REFERENCIAS

- [1] Documentación de Pygame.  
Disponible en: <https://www.pygame.org/docs/>
- [2] Tutorial sobre Pygame.  
Disponible en: <https://pygame.readthedocs.io/en/latest/index.html>
- [3] Tutorial para hacer las damas en Pygame (Parte 1). Disponible en: <https://www.youtube.com/watch?v=vnd3RfeG3NM&list=PLzMcbGfZo4-lkJr3sqpikNyVzbNZLRiT3>
- [4] Tutorial para hacer las damas en Pygame (Parte 2). Disponible en: <https://www.youtube.com/watch?v=LSYj8GZMjWY&list=PLzMcbGfZo4-lkJr3sqpikNyVzbNZLRiT3&index=2>
- [5] Tutorial para hacer las damas en Pygame (Parte 3). Disponible en: <https://www.youtube.com/watch?v=kOXGzkbnp&list=PLzMcbGfZo4-lkJr3sqpikNyVzbNZLRiT3&index=3>
- [6] Tutorial para hacer el menú en Pygame. Disponible en: <https://www.youtube.com/watch?v=GMBqjxcKogA>
- [7] Código del componente "Button". Disponible en: <https://github.com/baraltech/Menu-System-PyGame/blob/main/button.py>
- [8] S. J. Russell and P. Norvig, "Adversarial Search and Games" in *Artificial intelligence: a modern approach*, Fourth edition. Hoboken, NJ: Pearson, 2021, pp. 146-173
- [9] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis and S. Colton: "A survey of Monte Carlo Tree Search Methods", *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1-8, 2012, doi: 10.1109/TCIAIG.2012.2186810.
- [10] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalch-brenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484-489, 2016, doi: 10.1038/nature16961.
- [11] [https://www.researchgate.net/figure/Monte-Carlo-Tree-Search-steps-5\\_fig2\\_316451377](https://www.researchgate.net/figure/Monte-Carlo-Tree-Search-steps-5_fig2_316451377)